ITAI-1371 - Intro to Machine Learning
Reflective Journal

Camila Ferreira da Silva

The Lab from module 9 was focused on Ensemble Methods, that is when we combine multiple individual models (often called "weak learners") into a single model, more accurate, and reliable predictive model. I was exposed to the training and comparison of a single Decision Tree and the Random Forest.

The first part of the lab was called: "The Wisdom of the Crowd" which is really interesting to search about and find out that this an expression we use to refer to an idea that the combined judgment, opinions, or estimates of a large and diverse group of people can be more accurate than the decision of a single expert, also known as collective intelligence. This concept is implemented to Machine Learning through Ensemble methods, not only one model but a committee working together. The two popular ensemble techniques addressed in this lab were Bagging (Bootstrap Aggregating) where we can train multiple models aka Decision Trees. And, Boosting where we train models sequentially, each new model focuses on correcting the error made by the previous one.

The most known boosting models are Gradient Boosting and AdaBoost. Adaptive Boost assigns equal weight to all training samples at the start and after each weak learner, it increases the weights of misclassified points and decreases the weights of correctly classified points. Gradient Boosting (GBM), on the other hand, starts with a base prediction and then adds models sequentially to reduce the errors made by previous predictions.

For the setup we used the Iris dataset to classify flower species. This dataset was collected by Edgar Anderson, but it was known because of the analysis of a biologist and statistician named Ronald Fisher. Contain 150 samples of Iris flowers shared in 3 species: *Iris setosa*, *Iris versicolor* e *Iris virginica*, 50 of each one.
These lines do not change or modify the dataset, they only prepare the dataset.

`Import pandas as pd:` is used to organize and analyze data from a table.
`from sklearn.datasets import load_iris:` it brings the dataset **Iris**, with informations about flowers. The dataset only can charge when we call this function later on.
`from sklearn.model_selection import train_test_split:` this is the step where we import the function that separate the data in two groups:
Training: used to teach the model; **Testing**: used to verify how the module performs with data that it was not used to teach it.
`from sklearn.tree import DecisionTreeClassifier:` Import the model of **Decision Tree**. It learns the rules from the data to classify each flower.
`from sklearn.ensemble import RandomForestClassifier:` Import the model **Random Forest**. It combines various Decision Trees and uses its results to make a classification.
`from sklearn.metrics import accuracy_score:` Import a function that calculates the accuracy: the proportions of the predictions corrected made by the model.

Now, we call the function **load_iris()** that we imported before and carry the dataset Iris. The results are stored in the variable **iris**.

```
iris = load_iris()
```

```
X = iris.data
y = iris.target
```

The X keeps the flower characteristics that are going to be used to make the predictions. comprimento e largura da sépala;
comprimento e largura da pétala.
Each row represents a flower, and each column represents a characteristic.
Stored in `y` the correct classification of each flower — each of the three species of Iris it belongs to. `y` is what the model will try to predict.

`X` contains the clues of each flower; `y` contains the right answer.

```
X_train, X_test, y_train, y_test = train_test_split(
```

We call the function train_test_split() para split the data. It returns four parts:

- X train: features used to train;
- X test: features reserved to test;
- y train: correct answers equivalent to the training data;
- y test: correct answers equivalent to the testing data..

The features and the answers need to be split together to continue the equivalence to the same flower.

```
    X,
    y,
    test_size=0.3,
```

Choosing **30% of the testing data**. The other 70% will be used to training. Since Iris has 150 flowers, it results in 105 for training and 45 for testing.

```
    random_state=42
)
```

Define one see to the random division. Then, to execute the code again, you obtain the same division. The number 42 could be different; the important thing is to keep the same number to reproduce the result.
In Part 3 we trained a Decision Tree, to make predictions for the training test and calculate how many predictions were corrected.
The model achieved perfect accuracy on this small test set. However, obtaining a 100% score does not guarantee that a single Decision Tree will generalize well to new data. Individual trees tend to memorize noise in the training set, a classic symptom of *overfitting*. For this reason, evaluating a Random Forest ensemble alongside the base model helps demonstrate how combining multiple trees produces a more stable predictive model.

In the part 4, there was the same process from part 3, but now using the Random Forest. Each tree gives a vote, and the forest chooses the species with more votes. The **test set**, both models got everything accurate: **100%** for consequence the **0.00%** was the difference.

- **The 10 fold cross validation**, the decision tree was **95.33%** and a random forest, **96.00%**. The random forest **0.67** percentage points higher in cross-validation.

On the cross validation, the data is divided into 10 parts. The model is trained and validated 10 times, using a different part to validate each time. The accuracy shown is the average across these runs. This helps assess how the model performs across different data splits.

However, is not a contradiction, the test accuracy being 100% and the average of the cross validation being smaller: are validations with different divisions. In this exercise, both models were perfect on the specific test, but the Random Forest had an average performance slightly better than the cross validation.

The last part was Feature Importance: a great benefit of tree-based ensembles like Random Forest is that they can tell you which features were most important for making accurate predictions. The assigned task was to extract and plot the feature importances from the trained Random Forest model. The Random Forest identified petal width and petal length as the most important features for classifying the Iris flowers. Sepal length had lower importance, while sepal width was the least important feature.

Danh Nguyen

My part was training the Random Forest and comparing it with the Decision Tree. I expected the forest to win, but both scored 100% on the small test set. Using 10-fold cross-validation, the Random Forest scored slightly higher (96.0% vs. 95.3%). This connected to Lab 8: a Random Forest reduces the high variance of a single tree by letting many trees vote.

I also hit an error in the shared notebook: my cell said “accuracy_tree is not defined” because leftover template text broke the Decision Tree cell. It showed me that one broken step stops everything that depends on it.

Great Inioluwa Olayiwola

Understanding the function behind the code steps of the machine learning workflow helps demystify what were once stressful, overwhelming commands. Each part of the workflow loading data, fitting the model, testing, and others has a role to play to keep the data clean. Loading and holding back data from the test set for the first time is equivalent to the real world case. The goal here is to determine whether the model is generalizing and not memorizing.

In supervised learning, a student learns using an answer key. The inputs (features matrix (X)) and the target (vector (y)) are given to the model during the training process to determine the rules and the mapping function. In the testing phase, the targets are removed to evaluate the model using only the learned mapping function. Unsupervised learning models learn the structure of the data and cluster them, since there is no target provided.

Decision trees have a lower variance when using ensembles, or the Random Forest approach. The downside of individual decision trees is the variance and their tendency to fit and overfit noise. With Random Forests, many different trees are grown on many different, smaller data sets and feature sets, resulting in trees with variance that cancels each other out, making for a much more stable and accurate model.

In addition to all of the previous tasks, interpreting ensemble model results helps reveal new domain-specific insights. For the Iris classification task, it was found that nearly all predictions were made using petal measurements and only a very small number of decisions were based on the sepal measurements. It was shown that statistical findings correspond to the world of botany in the fact that petal sizes tend to vary a lot between different species, showing that machine learning is less about creating syntax for code and more about finding the trends and patterns.

Karima Sadruddin

Through this lab, I learned how a Decision Tree works and how machine learning models can be trained to classify data. At first, I found it a little challenging to understand how to write the code and connect the training process with making predictions. However, by following the steps and practicing with the Iris dataset, I became more comfortable using DecisionTreeClassifier and accuracy_score.

One thing I learned was how important it is to evaluate a model using test data to understand how well it performs on new information. My Decision Tree achieved 100% accuracy, which was a great result. I also learned that a single Decision Tree can sometimes overfit, which is why ensemble methods such as Random Forest are useful. Overall, this lab improved my understanding of machine learning, model training, and evaluation. I feel more confident working with Python and understanding how different machine learning models make predictions.

Quan Thach

This lab introduced me to ensemble methods, and the idea of a crowd of models voting on one answer was easy to picture once I connected it to asking many people the same question. The notebook explained bagging and boosting, and it showed that a Random Forest is built from many Decision Trees that each see different parts of the data and features. Before this lab, I thought of a model as one thing that learns one set of rules. Seeing a model described as a committee changed how I think about making predictions more reliable.

The most surprising part was the result. I expected the Random Forest to clearly beat the single tree, but both reached 100% accuracy on the 45 test flowers. At first that felt like the lab did not prove its point. After thinking about it more, I understood that the Iris dataset is small and the species are well separated, so even one tree can do a perfect job. That taught me that a method being stronger in general does not always show up on every dataset, and that I should be careful about drawing big conclusions from one easy test set.

The feature importance part was the most interesting to me. The Random Forest showed that petal width and petal length mattered far more than sepal length and sepal width. That matched what I would guess about the flowers, and it showed me that a model can explain what it relies on, not just give answers. I liked that this gave me a way to check whether the model's logic made sense instead of just trusting the accuracy number.

If I did this lab again, I would test the models on a harder or noisier dataset, or try a few different random splits, so that the difference between a single tree and a forest could actually

be seen. I would also finish the code cells earlier so the results are visible in the notebook itself. Overall, the lab gave me a clear picture of why ensembles are useful, how to read feature importance, and why it is worth questioning a result that looks too perfect.